Automated Academic Process Validator

Web Development Phase 2

Student: HARISURYAPRAKASH O

Register No: 7376231EC159

Department: Electronics and Communication Engineering

Date: 10 March 2026

Backend API Development

Developed a robust backend infrastructure using **Node.js and Express.js** to power the academic validation system. Implemented RESTful APIs that handle all core system operations, from user authentication to assignment management and validation workflows.

APIs rigorously tested using **Postman** to ensure reliability, proper error handling, and correct data formatting across all endpoints.

Authentication Endpoints

```
POST /api/auth/login
POST /api/auth/register
```

Assignment Endpoints

```
POST /api/assignments/create
GET /api/assignments/student
GET /api/assignments/faculty
```

Submission Endpoints

```
POST /api/submissions/upload
GET /api/submissions/student
```

Database & Authentication Integration

MongoDB Structure

Primary database with four collections: Users, Assignments, Submissions, and ValidationLogs

JWT Authentication

Secure token-based authentication using JSON Web Tokens for session management

Password Security

bcrypt encryption for secure password storage and retrieval

Authentication Flow

1. **User Login**
   Credentials submitted
2. **Server Verification**
   Authentication check
3. **Token Generation**
   JWT created
4. **Protected Access**
   API authorization

Full Stack CRUD Operations

Implemented complete Create, Read, Update, and Delete operations across the entire application stack, enabling seamless interaction between faculty, students, and the system.

Create Operations

- Faculty creates assignments with validation rules
- Students submit completed assignments

Read Operations

- Students view assigned tasks and deadlines
- Faculty review all submissions

Update Operations

- Faculty modify assignment rules
- Students update submissions

Delete Operations

- Remove assignments or submissions
- System cleanup functionality

State Management

React Hooks Implementation

Utilised React **Hooks** for efficient state management throughout the application. This approach ensures optimal UI updates and controlled data flow while minimising unnecessary re-renders.

Key Hooks Used

- `useState` – Manages component state including user data, assignment details, and submission results
- `useEffect` – Handles side effects and data fetching operations

State Data Storage

- Currently logged-in user information
- Assignment data and metadata
- Submission results and validation outcomes

Error Handling & Security

Error Handling

- API error responses with proper HTTP status codes
- Try-catch blocks in backend controllers
- User-friendly error messages

Security Measures

- JWT authentication for all protected routes
- Password hashing with bcrypt
- Input validation and sanitisation
- Environment variables for sensitive data

Example Security Check

```
if (!token) {
 return res.status(401).json({
 message: "Unauthorized"
 });
}
```

API Testing

Testing Tools & Methodology

Comprehensive API testing conducted using **Postman** and **Thunder Client** to verify endpoint functionality, response formats, and error handling.

APIs Tested

- Login API — Authentication verification
- Assignment creation API — Faculty workflow
- Submission API — Student upload process
- Validation API — Rule engine execution

Testing ensured API reliability and debugged backend functionality before frontend integration.

System Demonstration

The complete workflow demonstrates how faculty and students interact with the automated validation system.

Create Assignment

Faculty defines tasks with validation rules

Student Assignment

System distributes tasks to relevant students

Student Submission

Students upload completed assignments

Rule Validation

Automated engine validates submission against rules

Result Display

System shows Completed or Rejected status

Future Improvements

The system provides a solid foundation with opportunities for enhanced functionality and improved user experience.

Plagiarism Detection

Integrate text-matching algorithms to identify copied content in student submissions

Cloud Storage

Implement cloud-based file storage for scalable and reliable submission handling

Email Notifications

Automated email alerts for assignment deadlines, submission confirmations, and validation results

Analytics Dashboard

Advanced reporting with visualisations showing submission trends, validation statistics, and performance metrics